SIMATS – Java Programming

Assignment 2 (CO2)

JAVA PROGRAMMING – ASSIGNMENT 2

Name: Dharshini J

Register Number: 192324059

Course Code: CSA0922

Department: Artificial Intelligence and Data Science

College: SIMATS Engineering, Chennai

GitHub Repository Link: https://github.com/Dharshiniramdj/java2026

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer

1. Aim

To compare String, StringBuilder and StringBuffer in Java by generating railway e-tickets and measuring their execution time, memory efficiency and performance under repeated string operations.

2. Theory

String: String is immutable. Once created, its value cannot be changed. Every concatenation using the + operator creates a new String object.

StringBuilder: StringBuilder is mutable. It allows modifications without creating a new object for every operation. It is faster for repeated string concatenation in single-threaded applications.

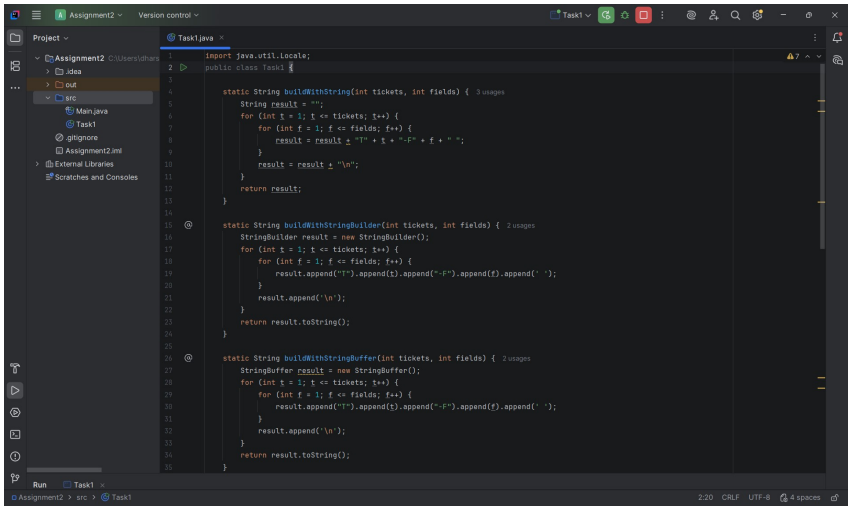
StringBuffer: StringBuffer is also mutable, but its methods are synchronized. It provides thread safety when multiple threads access the same buffer.

3. Program Code

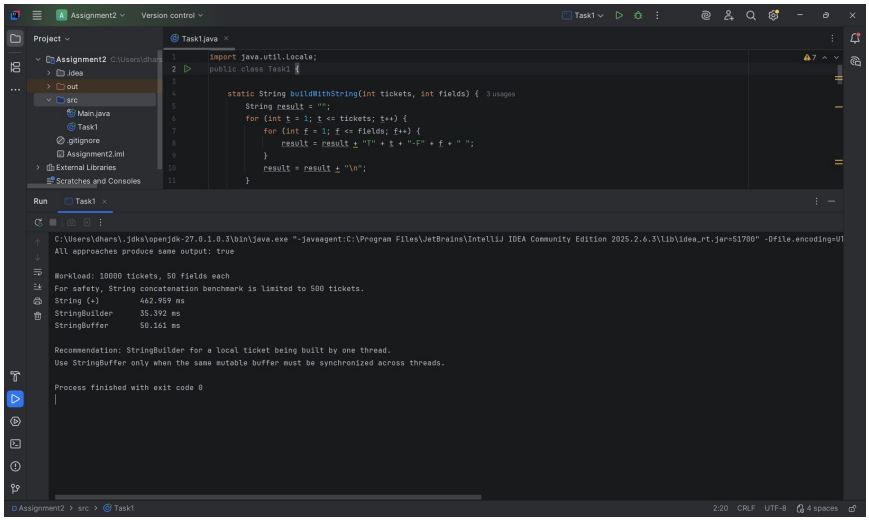
```
public class Main {
    public static void main(String[] args) {
        // String
        String s = "";
        long start = System.nanoTime();
        for (int i = 1; i <= 5; i++) {
            s = s + "Ticket " + i + "\n";
        }
        long end = System.nanoTime();
        System.out.println("String Output:");
        System.out.println(s);
        System.out.println("Time: " + (end - start) + " ns");
        // StringBuilder
        StringBuilder sb = new StringBuilder();
        start = System.nanoTime();
        for (int i = 1; i <= 5; i++) {
```

```
            sb.append("Ticket ").append(i).append("\n");
        }
        end = System.nanoTime();
        System.out.println("StringBuilder Output:");
        System.out.println(sb);
        System.out.println("Time: " + (end - start) + " ns");
        // StringBuffer
        StringBuffer sf = new StringBuffer();
        start = System.nanoTime();
        for (int i = 1; i <= 5; i++) {
            sf.append("Ticket ").append(i).append("\n");
        }
        end = System.nanoTime();
        System.out.println("StringBuffer Output:");
        System.out.println(sf);
        System.out.println("Time: " + (end - start) + " ns");
    }
}
```

4. **Screenshot of code executed in compiler**



5. Screenshot of output



Q2 – When Is ArrayList an Appropriate Engineering Choice?

1. Aim

To compare ArrayList, LinkedList and ArrayDeque based on insertion, deletion, searching and accessing operations to identify the most suitable collection for playlist management.

2. Theory

ArrayList: Uses a dynamic array to store elements. It provides fast random access but inserting or deleting elements in the middle requires shifting.

LinkedList: Uses nodes connected through links. Insertion and deletion are efficient when the required position is already reached, but accessing elements by index is slower.

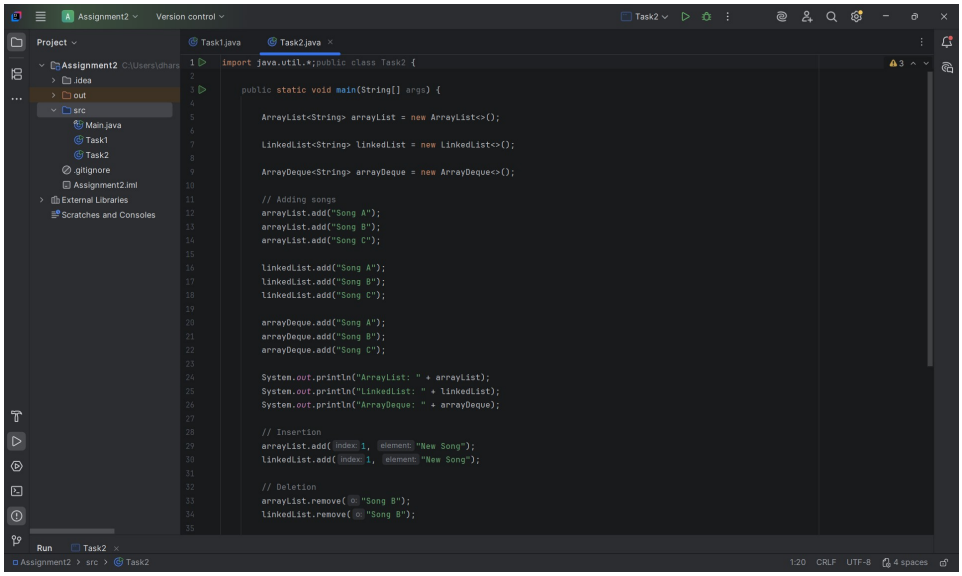
ArrayDeque: Uses a resizable array to implement a double-ended queue. It allows efficient insertion and deletion at both ends.

3. Program Code

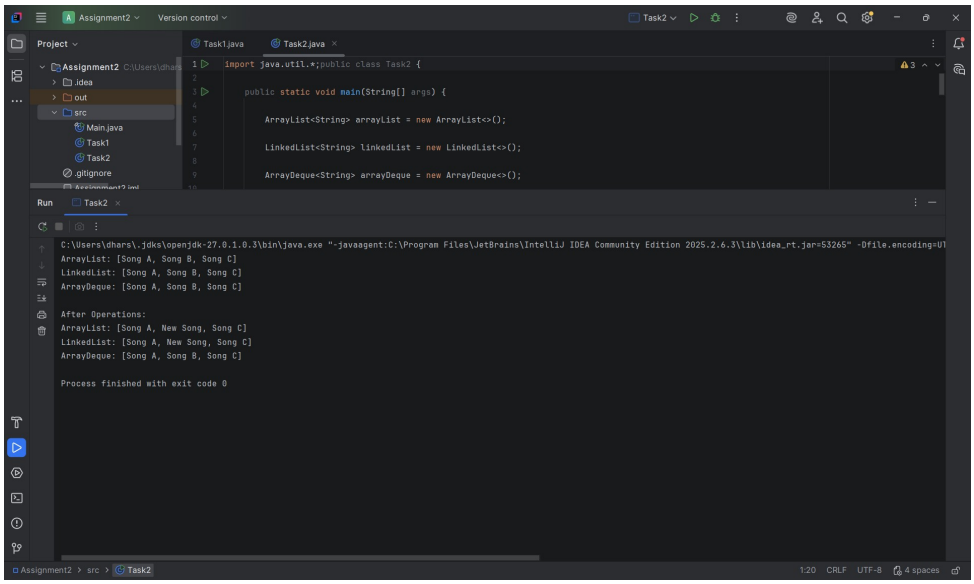
```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        ArrayList<String> arrayList = new ArrayList<>();
        LinkedList<String> linkedList = new LinkedList<>();
        ArrayDeque<String> arrayDeque = new ArrayDeque<>();
        // Adding songs
        arrayList.add("Song A");
        arrayList.add("Song B");
        arrayList.add("Song C");
        linkedList.add("Song A");
        linkedList.add("Song B");
        linkedList.add("Song C");
        arrayDeque.add("Song A");
        arrayDeque.add("Song B");
        arrayDeque.add("Song C");
        System.out.println("ArrayList: " + arrayList);
        System.out.println("LinkedList: " + linkedList);
```

```
        System.out.println("ArrayDeque: " + arrayDeque);
        // Insertion
        arrayList.add(1, "New Song");
        linkedList.add(1, "New Song");
        // Deletion
        arrayList.remove("Song B");
        linkedList.remove("Song B");
        // Deque operations
        arrayDeque.addFirst("First Song");
        arrayDeque.addLast("Last Song");
        arrayDeque.removeFirst();
        arrayDeque.removeLast();
        System.out.println("\nAfter Operations:");
        System.out.println("ArrayList: " + arrayList);
        System.out.println("LinkedList: " + linkedList);
        System.out.println("ArrayDeque: " + arrayDeque);
    }
}
```

4. Screenshot of code executed in compiler



5. Screenshot of output



Q3 – Engineer the hashCode-equals Contract

1. Aim

To understand and implement the equals() and hashCode() methods in Java to identify duplicate student attendance records using HashSet.

2. Theory

HashSet is a collection that stores unique elements and does not allow duplicates.

The **equals()** method checks whether two objects are logically equal.

The **hashCode()** method generates an integer value used by hash-based collections to identify the storage location of objects.

Important rules:

- If two objects are equal using equals(), they must have the same hash code.
- Different objects may have the same hash code.
- Both methods should be overridden together.

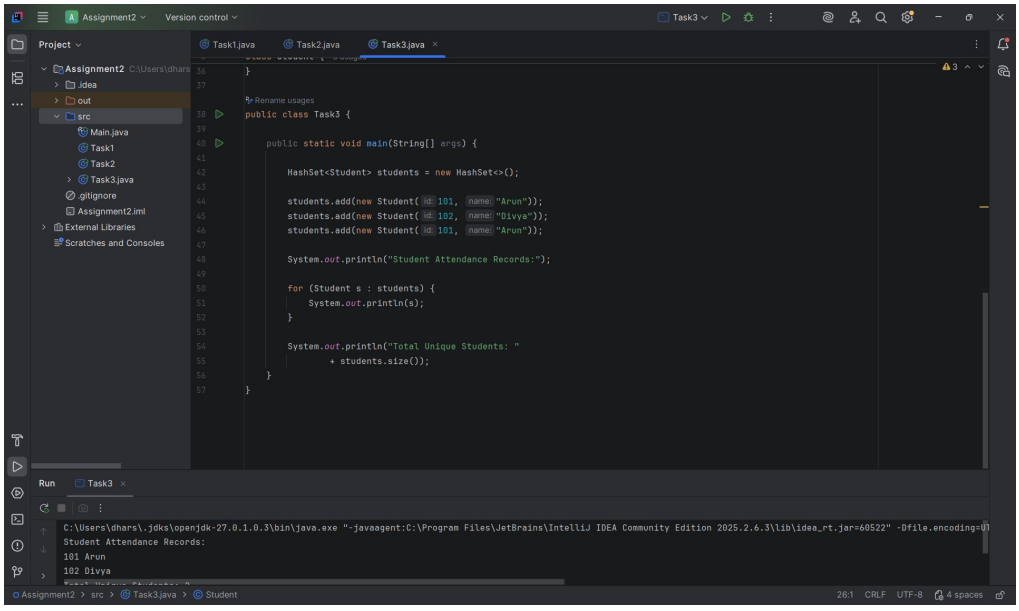
◆ HashSet uses hashCode() and equals() to identify duplicates.

3. Program Code

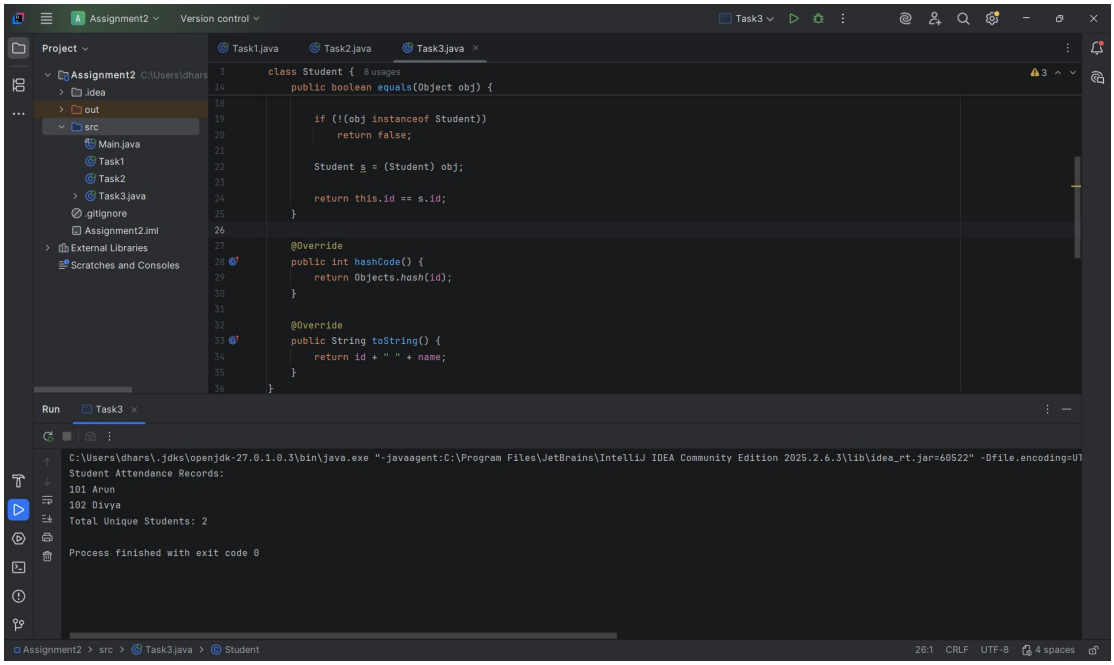
```
import java.util.*;
class Student {
	int id;
	String name;
	Student(int id, String name) {
		this.id = id;
		this.name = name;
	}
	@Override
	public boolean equals(Object obj) {
		if (this == obj)
			return true;
		if (!(obj instanceof Student))
			return false;
		Student s = (Student) obj;
		return this.id == s.id;
	}
	@Override
	public int hashCode() {
		return Objects.hash(id);
	}
	@Override
	public String toString() {
		return id + " " + name;
	}
}
```

```
public class Main {
    public static void main(String[] args) {
        HashSet<Student> students = new HashSet<>();
        students.add(new Student(101, "Arun"));
        students.add(new Student(102, "Divya"));
        students.add(new Student(101, "Arun"));
        System.out.println("Student Attendance Records:");
        for (Student s : students) {
            System.out.println(s);
        }
        System.out.println("Total Unique Students: "
                + students.size());
    }
}
```

4. Screenshot of code executed in compiler



5. Screenshot of output



Q4 – Choosing a Map for a Pharmacy Inventory

1. Aim

To compare HashMap, TreeMap and LinkedHashMap for storing and retrieving medicine details and to identify the appropriate map implementation for pharmacy inventory management.

2. Theory

HashMap: Stores data in key-value pairs. It provides average O(1) insertion and searching but does not guarantee ordering.

TreeMap: Stores keys in sorted order. It provides O(log n) insertion, deletion and searching.

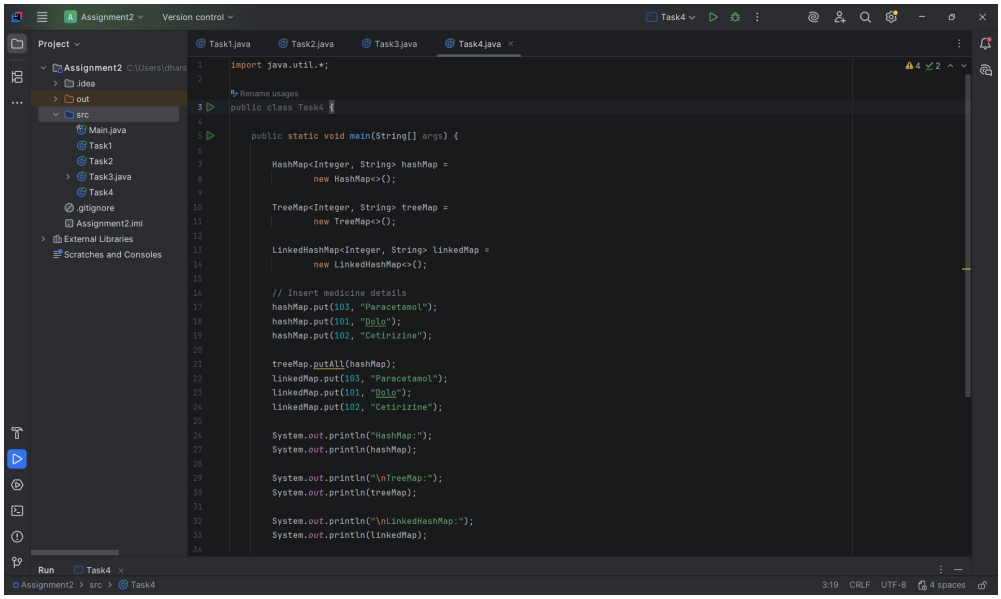
LinkedHashMap: Maintains insertion order while storing key-value pairs. It provides average O(1) lookup.

3. Program Code

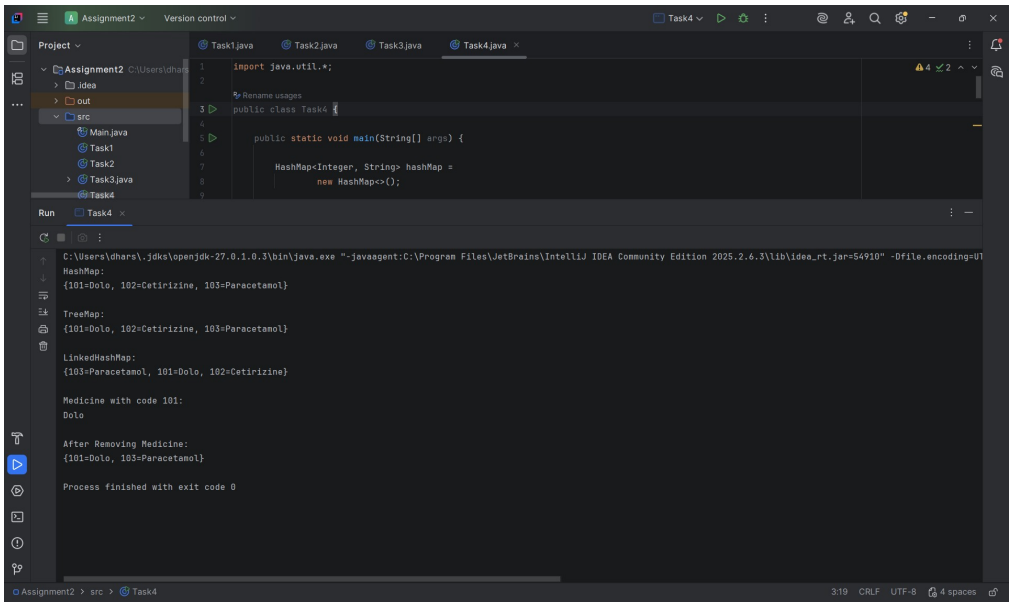
```
import java.util.*;
public class Main {
	public static void main(String[] args) {
		HashMap<Integer, String> hashMap =
				new HashMap<>();
		TreeMap<Integer, String> treeMap =
				new TreeMap<>();
		LinkedHashMap<Integer, String> linkedMap =
				new LinkedHashMap<>();
		// Insert medicine details
		hashMap.put(103, "Paracetamol");
		hashMap.put(101, "Dolo");
		hashMap.put(102, "Cetirizine");
		treeMap.putAll(hashMap);
		linkedMap.put(103, "Paracetamol");
		linkedMap.put(101, "Dolo");
		linkedMap.put(102, "Cetirizine");
		System.out.println("HashMap:");
		System.out.println(hashMap);
		System.out.println("\nTreeMap:");
		System.out.println(treeMap);
		System.out.println("\nLinkedHashMap:");
		System.out.println(linkedMap);
		// Searching
		System.out.println("\nMedicine with code 101:");
		System.out.println(hashMap.get(101));
		// Removing
```

```
        hashMap.remove(102);
        System.out.println("\nAfter Removing Medicine:");
        System.out.println(hashMap);
    }
}
```

4. Screenshot of code executed in compiler



5. Screenshot of output



Q5 – Derive a Generic Ranking Component from a Principle

1. Aim

To develop a reusable generic leaderboard using Java Generics and Comparator to rank teams based on points, goal difference and team name.

2. Theory

Generics: Generics allow classes and methods to work with different data types while providing type safety.

Comparable: Comparable defines the natural ordering of objects using the compareTo() method.

Comparator: Comparator defines custom ordering using the compare() method.

Comparator chaining: Multiple conditions can be combined to sort objects based on different criteria.

For the leaderboard:

Higher points receive higher ranking.

If points are equal, higher goal difference receives higher ranking.

If both are equal, team names are sorted alphabetically.

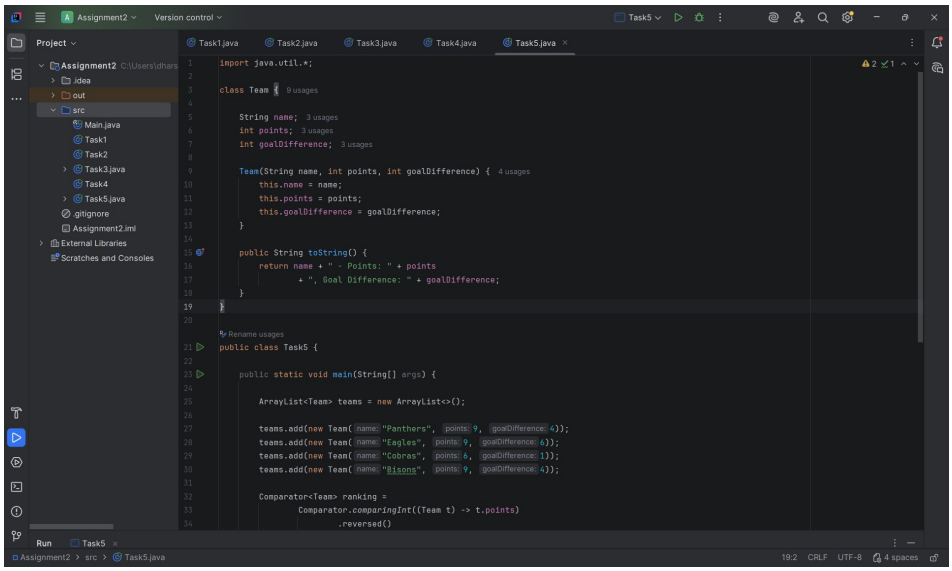
3. Program Code

```
import java.util.*;
class Team {
```

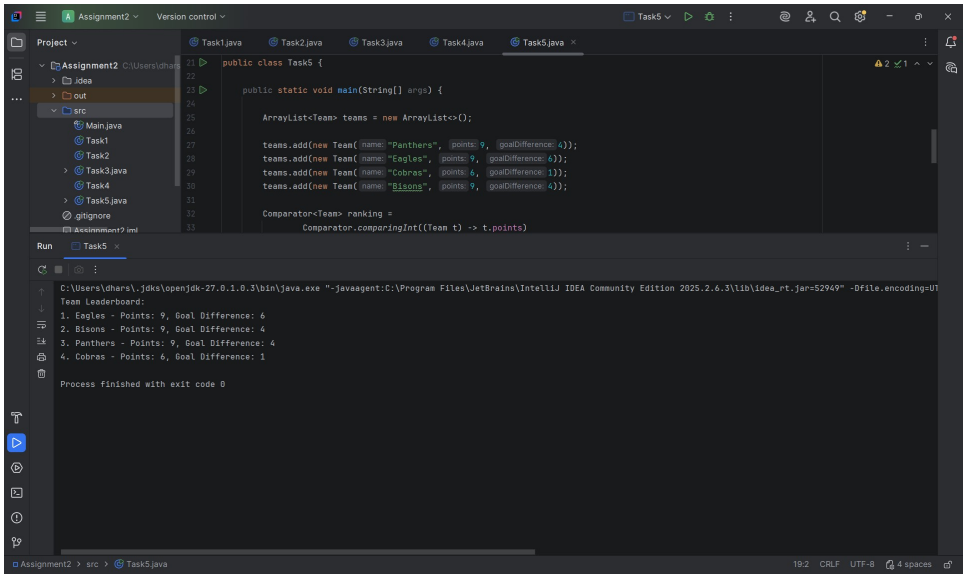
```
        String name;
        int points;
        int goalDifference;
        Team(String name, int points, int goalDifference) {
                this.name = name;
                this.points = points;
                this.goalDifference = goalDifference;
        }
        public String toString() {
                return name + " - Points: " + points
                                + ", Goal Difference: " + goalDifference;
        }
}
public class Main {
        public static void main(String[] args) {
                ArrayList<Team> teams = new ArrayList<>();
                teams.add(new Team("Panthers", 9, 4));
                teams.add(new Team("Eagles", 9, 6));
                teams.add(new Team("Cobras", 6, 1));
                teams.add(new Team("Bisons", 9, 4));
                Comparator<Team> ranking =
                                Comparator.comparingInt((Team t) -> t.points)
                                .reversed()
                                .thenComparing(
                                        Comparator.comparingInt(
                                                (Team t) -> t.goalDifference
                                        ).reversed()
                                )
                                .thenComparing(t -> t.name);
```

```
teams.sort(ranking);
System.out.println("Team Leaderboard:");
int rank = 1;
for (Team t : teams) {
        System.out.println(rank + ". " + t);
        rank++;
}}}
```

4. Screenshot of code executed in compiler



5. Screenshot of output



Q6 – Priority, Order and Fairness in a Queue

1. Aim

To implement a hospital emergency triage system using PriorityQueue to treat patients according to severity while maintaining arrival order among patients with equal severity.

2. Theory

A Queue follows the FIFO (First In First Out) principle. A PriorityQueue processes elements according to their priority rather than their insertion order.

In a hospital emergency system:

Severity 1 represents critical patients.

Severity 2 represents high-priority patients.

Severity 3 represents moderate patients.

Severity 4 represents low-priority patients.

Severity 5 represents minor patients.

A Comparator is used to arrange patients according to severity. An arrival counter is used to maintain FIFO order among patients having equal severity.

Starvation: Lower-priority patients may wait indefinitely if higher-priority patients continuously arrive.

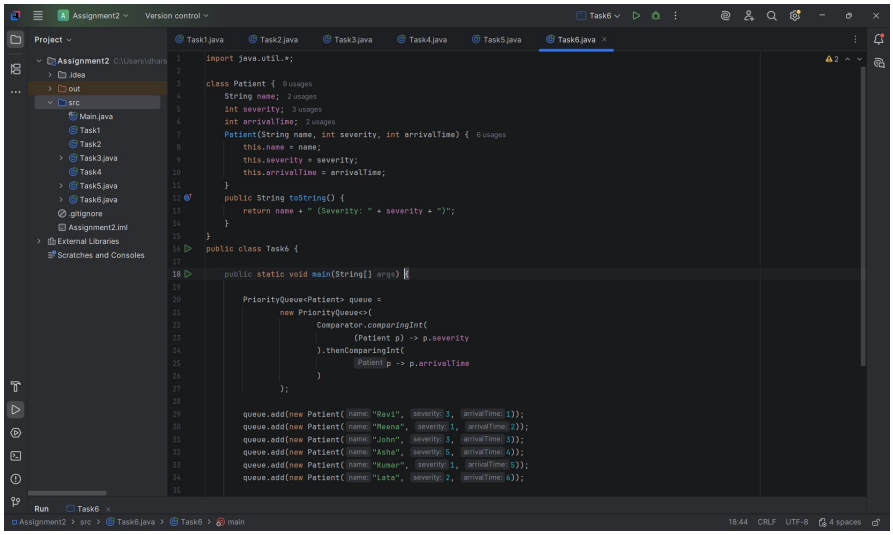
Solution: Priority aging can gradually increase the priority of patients who have waited for a long time.

4. Program Code

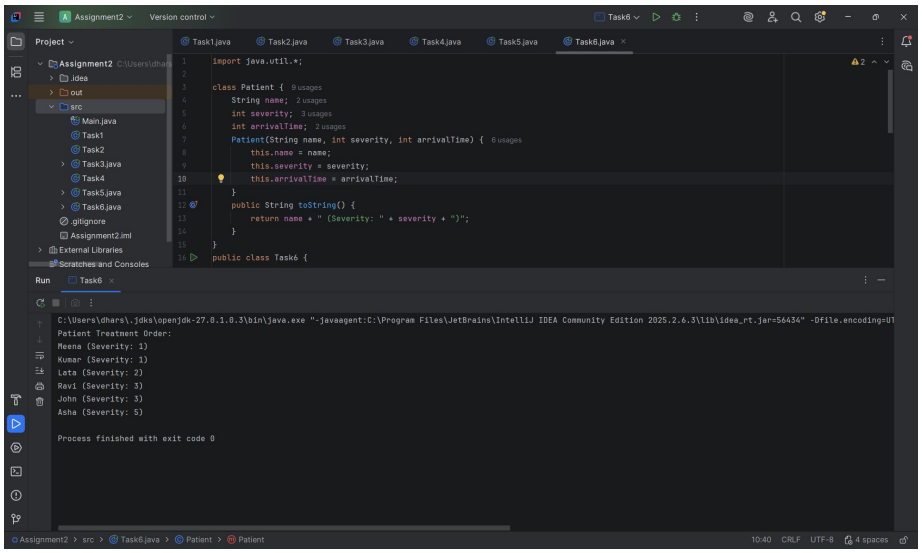
```
import java.util.*;
class Patient {
        String name;
        int severity;
        int arrivalTime;
        Patient(String name, int severity, int arrivalTime) {
                this.name = name;
                this.severity = severity;
                this.arrivalTime = arrivalTime;
        }
        public String toString() {
                return name + " (Severity: " + severity + ")";
        }
}
public class Main {
        public static void main(String[] args) {
                PriorityQueue<Patient> queue =
                        new PriorityQueue<>(
                                Comparator.comparingInt(
                                        (Patient p) -> p.severity
                                ).thenComparingInt(
                                        p -> p.arrivalTime
                                )
                        );
                queue.add(new Patient("Ravi", 3, 1));
                queue.add(new Patient("Meena", 1, 2));
                queue.add(new Patient("John", 3, 3));
```

```
        queue.add(new Patient("Asha", 5, 4));
        queue.add(new Patient("Kumar", 1, 5));
        queue.add(new Patient("Lata", 2, 6));
        System.out.println("Patient Treatment Order:");
        while (!queue.isEmpty()) {
            Patient p = queue.poll();
            System.out.println(p);
    }   }   }
```

4. Screenshot of code executed in compiler



5. Screenshot of output



Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate

1. Aim

To identify the performance issues caused by repeated String concatenation and redesign the program using StringBuilder to improve efficiency and reduce unnecessary object creation.

2. Theory

Repeated String concatenation using the + operator creates new String objects because String is immutable.

When a large amount of text is generated repeatedly, the existing content may be copied multiple times, increasing execution time and memory usage.

StringBuilder provides a mutable character sequence and allows efficient modification using append().

Time complexity:

Repeated concatenation can require $O(L^2)$ total copying.

StringBuilder append operations take amortized O(L) total time for output length L, excluding the final string conversion.

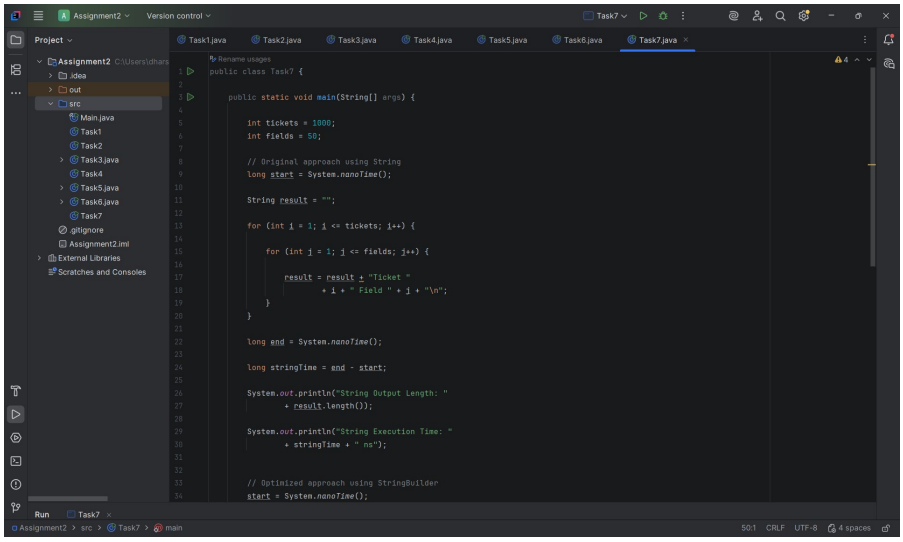
3. Program Code

```
public class Main {
    public static void main(String[] args) {
        int tickets = 1000;
        int fields = 50;
        // Original approach using String
```

```
long start = System.nanoTime();
String result = "";
for (int i = 1; i <= tickets; i++) {
        for (int j = 1; j <= fields; j++) {
                result = result + "Ticket "
                                + i + " Field " + j + "\n";
        }     }
long end = System.nanoTime();
long stringTime = end - start;
System.out.println("String Output Length: "
                + result.length());
System.out.println("String Execution Time: "
                + stringTime + " ns");
// Optimized approach using StringBuilder
start = System.nanoTime();
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= tickets; i++) {
        for (int j = 1; j <= fields; j++) {
                sb.append("Ticket ")
                  .append(i)
                  .append(" Field ")
                  .append(j)
                  .append("\n");
        }     }
end = System.nanoTime();
long builderTime = end - start;
System.out.println("\nStringBuilder Output Length: "
                + sb.length());
System.out.println("StringBuilder Execution Time: "
```

```
                + builderTime + " ns");
        System.out.println("\nPerformance Comparison:");
        System.out.println("String: "
                + stringTime + " ns");
        System.out.println("StringBuilder: "
                + builderTime + " ns");
        System.out.println("\nOptimization completed.");
    }       }
```

4. Screenshot of code executed in compiler



5. Screenshot of output

